

HANDS-ON LAB

Build a Full Stack Banking Application with Bob



Intermediate
Difficulty

~60 Minutes
Duration

7 Prompts

LAB OVERVIEW

DIFFICULTY	DURATION	AI TOOL
Intermediate	~60 minutes	IBM Bob

TECH STACK

HTML + Bootstrap	Python Flask	SQLite	REST APIs
------------------	--------------	--------	-----------

APPLICATION FEATURES

✓ Customer Login	✓ Dashboard
✓ View Balance	✓ Deposit Funds
✓ Withdraw Funds	✓ Logout

ARCHITECTURE

Frontend	Backend	Database
HTML pages with Bootstrap — login, dashboard, deposit & withdraw forms	Python Flask — routes, controllers, session management, validation logic	SQLite — lightweight, file-based, embedded in the backend folder

P1 Prompt 1: Architecture & Planning

~5 min

Ask the AI to produce a high-level implementation plan. No code is written yet — purely a planning document covering architecture, folder structure, module breakdown and a development roadmap.

Output: IMPLEMENTATION_PLAN.md

Prompt Text: (in Plan Mode)

```
Create a high-level implementation plan in Markdown (.md) named as
IMPLEMENTATION_PLAN.md for a Banking Web Application.
Technology:
• Frontend: HTML, Bootstrap (FRONTEND folder)
• Backend: Python Flask (BACKEND folder)
• Database: SQLite (in backend)
Features:
• Customer Login
• Dashboard
• View Balance
• Deposit Funds
• Withdraw Funds
• Logout
Generate a concise architecture and planning document.
Include:
1. Solution Overview
  • Objective
  • Scope
  • Users
  • Functional requirements
  • Non-functional requirements
  • Assumptions
2. High-Level Architecture
  • Architecture diagram
  • Frontend → Backend → Database interaction
  • Request lifecycle
3. Component Design
  • Frontend responsibilities
  • Backend responsibilities
  • Database responsibilities
4. Folder Structure
  • Project directory
  • Responsibility of each folder
5. Module Breakdown
  • Authentication
  • Dashboard
  • Account Management
  • Transactions
6. Implementation Roadmap
  • Development phases
  • Estimated effort
  • Dependencies
Keep this document at planning level only.
Do NOT generate database schema, SQL scripts, API contracts, or detailed
implementation steps
```

Covers:

- ✓ Solution overview (objective, scope, users, requirements)

- ✓ High-level architecture with request lifecycle
- ✓ Component design for frontend, backend and database
- ✓ Folder structure with responsibilities
- ✓ Module breakdown and implementation roadmap

Tip: No SQL, API contracts or code in this step. Planning only.

P2

Prompt 2: Step-by-Step Implementation Guide

~5 min

Reference the plan file and ask for detailed plain-English instructions covering every implementation layer — from environment setup to deployment.

Output: STEP_BY_STEP_IMPLEMENTATION_GUIDE.md

Prompt Text: (in Plan Mode)

```
@/IMPLEMENTATION_PLAN.md Refer
```

```
Generate step-by-step implementation instructions in md file named as  
STEP_BY_STEP_IMPLEMENTATION_GUIDE.md that  
Include:
```

1. Environment Setup
 - Install dependencies
 - Virtual environment
 - Flask setup
2. Backend Implementation
 - Create Flask app
 - Routes
 - Controllers
 - Services
 - Session management
 - Error handling
3. Frontend Implementation
 - Login page
 - Dashboard
 - Deposit form
 - Withdraw form
 - Bootstrap layout
4. Integration Steps
 - Connect frontend to APIs
 - Connect Flask to SQLite
5. Validation Rules
 - Login validation
 - Balance validation
 - Deposit/withdraw checks
6. Testing
 - Unit tests
 - Integration tests
 - Manual testing checklist
7. Deployment
 - Run locally
 - Production considerations

```
Generate plain english instructions & and logic of how to implement and not the  
entire code.
```

Covers:

- ✓ Environment setup — virtualenv, Flask, dependencies
- ✓ Backend implementation — routes, controllers, session handling
- ✓ Frontend implementation — login, dashboard, deposit & withdraw forms

- ✓ Integration steps, validation rules and testing checklist
- ✓ Deployment — local run and production considerations

Tip: Use @/IMPLEMENTATION_PLAN.md as context in your prompt.

P3 Prompt 3: Full Application Build

~10 min

The main build step. Reference the implementation guide and ask the AI to build the entire application end-to-end, module by module, following the defined architecture.

Output: FRONTEND/ BACKEND/ SQLite DB

Prompt Text: (auto switch to Agent Mode)

```
Refer to @/STEP_BY_STEP_IMPLEMENTATION_GUIDE.md and begin implementing the end-to-end full-stack banking application. Review the implementation guide thoroughly and follow the defined architecture, project structure, and development sequence. Build the application incrementally across all layers:
1. Frontend - Implement the UI components, pages, routing, state management, and API integration.
2. Backend - Develop APIs, business logic, authentication, validation, and service layers.
3. Database - Create schema, models, migrations, and data access layer.
4. Integration - Connect frontend, backend, and database end-to-end.
5. Security & Configuration - Apply authentication, authorization, environment configuration, and secure coding practices.
6. Testing - Add unit, integration, and functional tests.
7. Execution - Ensure the application runs locally with clear startup instructions.
Start with project setup and foundation components, then proceed module by module according to the implementation guide.
```

Covers:

- ✓ Frontend — UI pages, Bootstrap layout, form handling
- ✓ Backend — Flask APIs, business logic, authentication
- ✓ Database — schema, models, data access layer
- ✓ Integration — frontend, backend and SQLite connected end-to-end
- ✓ Security config and startup instructions included

Tip: Use @/STEP BY STEP IMPLEMENTATION GUIDE.md as context. This is the longest step.

---- Configuration of GitHub MCP and mcp.json in .bob directory ---- 10 mins

P4 Prompt 4: Push to GitHub

~10 min

Push the completed local application to a new GitHub repository using the GitHub MCP connection, excluding environment, cache and database files.

Output: GitHub repository: bob-banking-app

Prompt Text:

```
Using the GitHub MCP connection, push my local banking application to GitHub.
Look up my GitHub username from the authenticated token – do not guess it.
1. Create a new public repository on my GitHub account named: bob-banking-app
   - Description: "Banking web application built with Flask, SQLite and
   Bootstrap"
   - Initialise it with a README
2. Push all files from my local workspace to the main branch EXCEPT:
   - anything inside venv/
   - anything inside __pycache__/ folders
   - anything inside .bob/
   - anything inside .pytest_cache/
   - any file ending in .pyc
   - any file ending in .db
   - the docs/ folder
3. Confirm the repository URL once all files are pushed
```

Covers:

- ✓ New public GitHub repository created and initialised
- ✓ Local codebase pushed to the main branch
- ✓ Environment, cache and database files excluded
- ✓ Repository URL confirmed

Tip: Requires the GitHub MCP connection to already be authenticated.

P5 Prompt 5: Add CI/CD Pipeline

~5 min

Add the CI/CD workflow file to the repository using the GitHub MCP connection, using the pipeline definition saved locally during setup.

Output: .github/workflows/banking-app-ci.yml

Prompt Text:

```
Using the GitHub MCP connection, add the CI/CD pipeline file to my repository.
Look up my GitHub username from the authenticated token – do not guess it.
1. In the repository bob-banking-app on my GitHub account, create the file at
   this exact path: .github/workflows/banking-app-ci.yml
2. Use the contents of the file I have saved locally at:
   Docs/demo-setup/banking-app-ci.yml
3. Commit it with the message: "Add CI/CD pipeline"
4. Confirm the file is visible in the repository.
```

Covers:

- ✓ CI/CD workflow file added at the correct path
- ✓ Workflow content sourced from the local pipeline file
- ✓ Change committed with a clear message
- ✓ File visibility confirmed in GitHub

Tip: The CI/CD file must already exist locally at docs/demo-setup/banking-app-ci.yml.

P6**Prompt 6: Automated Code Change & Pull Request**

~10 min

Ask Bob to make a precise, scoped code change directly in the GitHub repository via MCP — adding validation checks to the withdraw route — then open a pull request for review.

Output: New branch, committed change and pull request

Prompt Text:

```
Using the GitHub MCP connection, make the following exact change
to my repository bob-banking-app on my GitHub account.
Look up my GitHub username from the authenticated token – do not guess it.
1. Create a new branch named: feature/withdraw-validation
2. Find the file in the repository that contains the withdraw route
   - it handles POST requests for withdrawing funds from a bank account
3. In that file, locate where the form amount is read from the request,
   and add these exact three validation checks immediately after that line,
   before any service or database call is made:
   - If the amount field is empty, return an error: "Amount is required"
   - If the amount is not a positive number, return an error:
     "Amount must be greater than zero"
   - If the amount exceeds the current balance, return an error:
     "Insufficient funds"
4. Do not change anything else in the file
5. Commit with message: "Add validation checks to withdraw route"
6. Create a pull request with:
   - base branch: main
   - compare branch: feature/withdraw-validation
   - title: "Add input validation to withdraw route"
   - body: "Adds three validation checks to the withdraw route to prevent
     invalid transactions in the banking application."
7. Confirm the PR number and URL
```

Covers:

- ✓ New feature branch created
- ✓ Three validation checks added to the withdraw route
- ✓ Change committed with a clear message
- ✓ Pull request opened with title, body and branches specified
- ✓ PR number and URL confirmed

Tip: Review the diff carefully before merging — this step edits code directly.

P7**Prompt 7: Automated Pull Request Review**

~5 min

Have Bob review the pull request it just created using the GitHub MCP connection, assessing validation completeness and security, and giving a recommendation.

Output: PR review comment and recommendation

Prompt Text:

```
Using the GitHub MCP connection, review the pull request you just created.
Look up my GitHub username from the authenticated token – do not guess it.
1. Read the open pull request on branch feature/withdraw-validation
   in repository bob-banking-app on my GitHub account
2. Review the changed code and tell me:
   - whether these three validation checks are sufficient for a
     banking withdrawal or if anything critical is missing
   - any security concerns with how the validation is implemented
   - whether you would approve or request changes, and why
```

3. Draft one inline review comment on the most important line in the change

Covers:

- ✓ Pull request read directly from GitHub
- ✓ Validation completeness and security assessed
- ✓ Approve / request-changes recommendation given
- ✓ Inline review comment drafted

Tip: Use this step to practice AI-assisted code review, not just generation.